HearDesk

End-to-End UI Test Report

Browser workflows • Recorded results • Screenshots

Website feedback. Ready for your team.

A personal full-stack engineering project

Prepared by Muhammad Danu Susyanto

Version 1.0 • 13 October 2026

Portfolio edition describing the implemented app, with test examples and dated results.

Contents

Contents 2
1. Purpose and recorded version 3
2. Test environment 3
3. How the tests were run 3
4. Features covered by the browser tests 4
5. UI test UC-01: send feedback and save tracking access 4
6. UI test UC-02: read team replies without seeing internal notes 4
7. UI test UC-03: dialogs, loading and screen sizes 5
8. Recorded scenario results 6
9. Screenshot: landing page 8
10. Screenshot: team inbox 9
11. Screenshot: visitor widget 10
12. What the results establish 10
13. How to maintain this report 11

1. Purpose and recorded version

This document explains the HearDesk features and code reviewed for this portfolio. The results describe the recorded version and test environment. They do not represent a customer contract, independent certification or measured business results.

End-to-End UI testing means using a real browser to follow a user workflow through the running application. Playwright performs actions such as filling the feedback form and checking the result on screen. These tests used a production build, meaning the compiled application intended for deployment, in a separate local test environment.

The run recorded here started on 13 October 2026 at 04:14:49.928 UTC. It tested Git commit afb4f54a83eafcd0f10232adbbeeae92292c293a. The result is historical evidence, not a new test execution caused by a wording update to this document.

The scenario table comes from the Playwright result file. A scenario is one named test and can contain several steps and checks. A check in test code is called an assertion.

2. Test environment

Table 1. Test environment

Item	Configuration and meaning
Test tool	Playwright 1.63.0 with Chrome on Windows. Headless means the browser ran without a visible desktop window.
Application	Next.js production build with E2E_PRODUCTION=true, served locally on port 3100.
Example website	A local website on port 3200 that embeds the widget.
Database	ureport_test, checked by the runner before data cleanup. The application database was not used.
Execution order	One worker runs tests sequentially. Some scenarios depend on records created by earlier scenarios.
Saved evidence	Structured result file, selected screenshots and failure traces when a test fails.
Email	A local test mailbox. No real email delivery is tested.
Source version	afb4f54a83eafcd0f10232adbbeeae92292c293a

3. How the tests were run

The application was built in .next-sonar, a separate build directory. The test runner checked its database target, applied test-database migrations, cleared previous test records and started both the application and example website. Tests used sample accounts and reports, not customer data.

Recorded command: HEARDESK_BUILD_DIR=.next-sonar E2E_PRODUCTION=true npm run test:e2e -- --reporter=json. On Windows, these environment variables are set before running npm. The JSON option saves results in a structured format that can be checked automatically.

Successful process exit codes and scenario results are separate checks. Log warnings also need review. A test fixture means the sample accounts, websites and reports created for the tests.

4. Features covered by the browser tests

Table 2. Features covered by the browser tests

Group	FSD IDs	What the tests check
Sign-in and setup	FR-01, FR-02	Create or verify a test account, open the right workspace and reject a session that has been revoked.
Visitor widget	FR-04 to FR-08	Open the embedded form, submit a report, attach an image, save tracking access and retry without a second report.
Inbox and settings	FR-03, FR-09 to FR-11	Search reports, use filters, save website settings, update status and add internal notes.
Private tracking	FR-12, FR-13	Open the private tracking page, read team replies and refresh status without exposing internal notes.
Company separation	FR-02, FR-06	Reject requests for another Company's reports, notes and screenshots without the required membership.
Reporting	FR-14	Compare the monthly chart and Excel output with stored report totals.
Platform management	FR-15	Use the separate service-management console and check records of supported administrative actions.
Screen behavior	FR-17	Check loading placeholders, errors, dialogs, reduced motion and layouts at different screen widths.

5. UI test UC-01: send feedback and save tracking access

Starting state: a registered test website belongs to an active Company. The visitor does not have a login session.

Steps: open the example website and its feedback widget. Enter a message, optionally attach a screenshot, and submit. Read the success message and save the private tracking link and code.

Expected result: one report is saved and the code has 48 hexadecimal characters. The link opens /track#code=… without requiring visitor login. Retrying the same identified request with unchanged content does not create a second report.

Related rejection checks cover changed content under a reused request identity, unsupported files and too many requests. The recorded scenario outcomes appear in the results table.

6. UI test UC-02: read team replies without seeing internal notes

Starting state: a tracked report exists and a test team account can update it.

Steps: send a reply to the visitor, add a separate internal note and set the report to REVIEWING. Open the tracking link in a visitor browser context, which has no team login. Check the reply and status, then set DONE in the team view and refresh the visitor page.

Expected result: the visitor sees the team reply and updated status, but cannot see the internal note. The link also opens in a new tab from the widget. An unknown code shows an error, and entering the correct code restores the report view.

The scenario checks page width on phone-sized and desktop screens. It uses separate team and visitor contexts so a team login cannot accidentally make the anonymous flow pass.

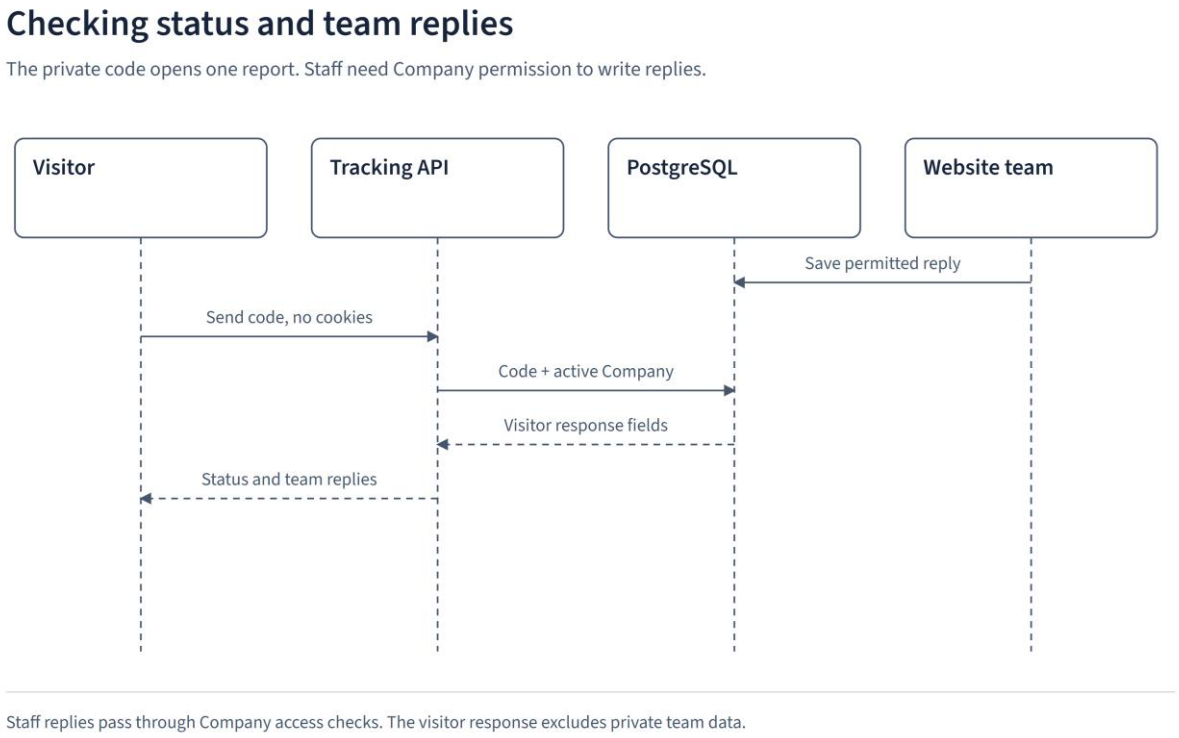


Figure 1. UI test UC-02: read team replies without seeing internal notes

7. UI test UC-03: dialogs, loading and screen sizes

Landing tests open and close the demo, press Escape, click outside the dialog and use the interactive preview. They also check that animations finish correctly after resizing and that reduced-motion preferences are respected.

Inbox tests check filters, website settings and loading placeholders. A skeleton is a temporary placeholder shown while a request is loading. The tests check that placeholders disappear when results or errors arrive.

Screenshots show appearance, while assertions check behavior. Passing in this Chrome environment does not establish full compatibility with every other browser or assistive technology.

8. Recorded scenario results

PASSED means that every assertion in that scenario succeeded during the recorded run. A skipped scenario did not run. A flaky result passes only after a failed attempt and retry. The recorded run had 19 passed scenarios, with no skipped, unexpected or flaky outcomes.

Exact test names are retained below so each row can be found in the source. Some names contain developer terms such as payload (the data sent in a request) or legacy routes (older API paths kept for compatibility). The preceding workflow table explains them in user terms.

Recorded start (UTC): 2026-10-13T04:14:49.928Z. Duration: 113.0 seconds. Results: 19 completed as expected, 0 unexpected outcomes, 0 skipped and 0 flaky.

Table 3. Recorded scenario results

ID / file	Executed scenario	Result / duration
E2E-01 **application.spec.ts**	successful setup with failed auto-login does not ask to create account again	PASSED 0.7 s
E2E-02 **application.spec.ts**	setup, private admin, anonymous widget, attachments and logout	PASSED 11.7 s
E2E-03 **application.spec.ts**	anonymous retries, conflicting payloads, invalid files and request limits	PASSED 2.3 s
E2E-04 **landing.spec.ts**	landing motion settles after rapid interaction and respects reduced motion	PASSED 1.9 s
E2E-05 **landing.spec.ts**	completed entrances remain settled across resizing and preference changes	PASSED 2.3 s
E2E-06 **landing.spec.ts**	widget illustration contents fit at intermediate widths	PASSED 0.4 s
E2E-07 **landing.spec.ts**	public landing demo never writes feedback and works on mobile	PASSED 2.7 s
E2E-08 **visual.spec.ts**	website list and add form align side-by-side on desktop and stack on mobile	PASSED 1.5 s
E2E-09 **visual.spec.ts**	desktop and mobile workspace render without overflow	PASSED 2.6 s
E2E-10 **visual.spec.ts**	search, filters, settings persistence and public widget configuration	PASSED 10.5 s
E2E-11 **visual.spec.ts**	widget key rotation requires confirmation and retires old key without deleting reports	PASSED 5.1 s
E2E-12 **zz-company-isolation.spec.ts**	Company API and legacy routes reject foreign reports, notes, attachments and platform-only access	PASSED 5.9 s
E2E-13 **zz-feedback-reporting.spec.ts**	Company monthly line chart and three Excel periods match aggregate data	PASSED 5.8 s
E2E-14 **zz-filter-ui.spec.ts**	filter popover floats without shifting table and preserves URL context	PASSED 2.7 s
E2E-15 **zz-platform.spec.ts**	platform owner has separate console, real metadata and audited Company management	PASSED 16.1 s
E2E-16 **zz-session-navigation.spec.ts**	landing and login recognize current owner session and revoked cookies	PASSED 2.9 s

ID / file	Executed scenario	Result / duration
E2E-17 **zz-skeleton.spec.ts**	data skeletons remain until response, disappear on success/error and respect reduced motion	PASSED 7.4 s
E2E-18 **zz-tracking.spec.ts**	visitor tracks an anonymous report and sees public replies but never internal notes	PASSED 6.2 s
E2E-19 **zzz-onboarding.spec.ts**	local client verifies email, creates Company, connects website and resets password	PASSED 12.5 s

9. Screenshot: landing page

Captured during the recorded browser run. The reports shown in the landing preview are examples, not customer submissions.

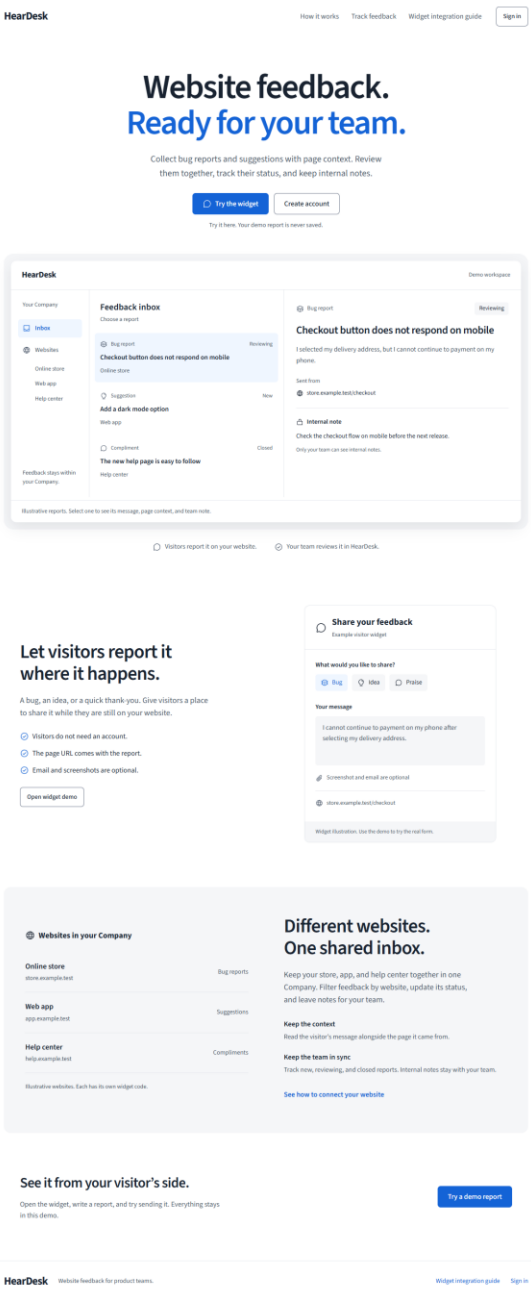


Figure 2. Test environment screenshot: landing page

10. Screenshot: team inbox

The inbox contains test reports in a separate test environment. It shows the screen used to review feedback and does not represent real customer activity.

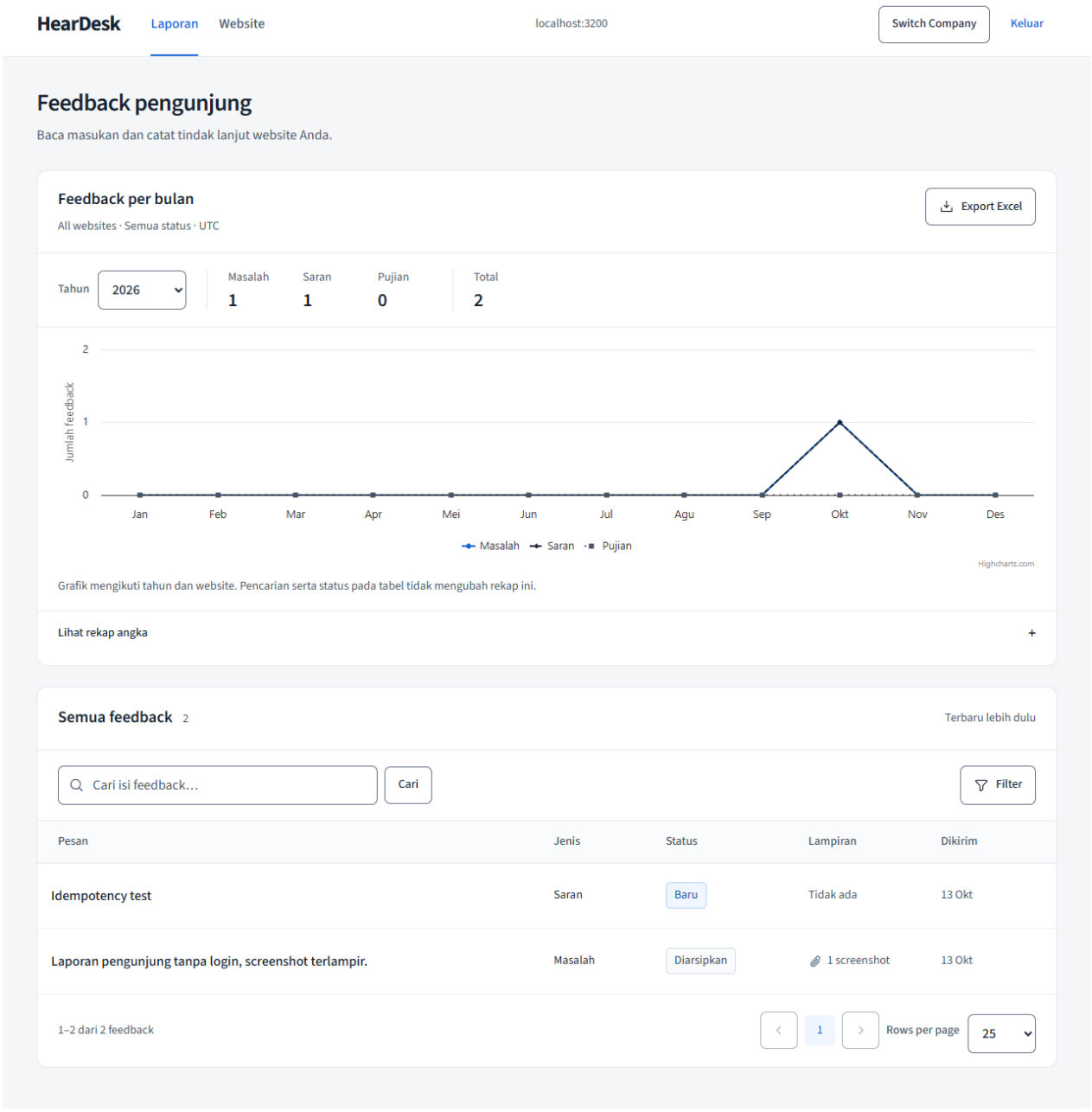


Figure 3. Test environment screenshot: team inbox

11. Screenshot: visitor widget

The registered test website displays the feedback form at a phone-sized screen width. The selected screenshot does not display a live private tracking code or a password.

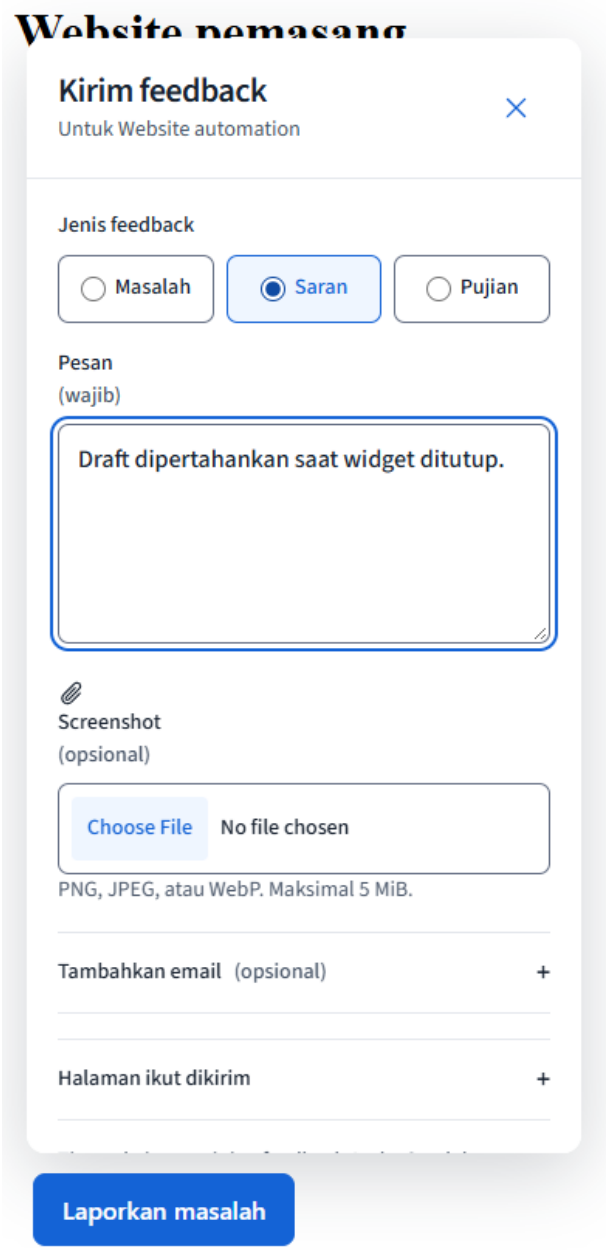


Figure 4. Test environment screenshot: visitor widget

12. What the results establish

All recorded scenarios passed on the stated version and environment. This verifies those workflows, not every possible use of the app. It does not measure heavy-load performance, prove that all vulnerabilities are absent, or establish production availability.

Earlier test logs contained warnings about concurrent pg queries and Vite configuration compatibility. A warning is different from a failed assertion and should still be reviewed. Some tests need the records produced by earlier tests, so running a single file may require additional setup.

Separate recorded checks passed 269 unit/component tests and 90 integration tests on 13 October 2026. Those counts are supporting evidence and are not included in the 19 browser scenarios.

13. How to maintain this report

Keep the test date, Git commit, browser, result file and screenshots together. When a feature changes, update its FSD requirement and test assertions before recording new results. Traces can include form data and are not automatically published.

Source files: tests/e2e/application.spec.ts, landing.spec.ts, visual.spec.ts, zz-company-isolation.spec.ts, zz-feedback-reporting.spec.ts, zz-filter-ui.spec.ts, zz-platform.spec.ts, zz-skeleton.spec.ts, zz-session-navigation.spec.ts, zz-tracking.spec.ts and zzz-onboarding.spec.ts.